{EPITECH}

CRASH & LEARN_

< MANDATORY SUBTITLE HERE />

Crash&Learn Grand Prix — Usage Instructions

🚀 Quick Start

```
# Build the Docker services
docker compose build

# Smoke test (runs demo.py)
docker compose run --rm sim
```

🧪 Tests & Demo

```
# Run the built-in demo (uses submission/agent.py if present, fallback controller otherwise)
docker compose run --rm sim

# Run contract unit tests
docker compose run --rm sim python test_contract.py

# Run integration physics tests
docker compose run --rm sim python test_integration.py
```




🏎️ Recording & Visualization (Record + Replay)

The simulation uses a **record-then-replay** workflow: `sim_recorder.py` simulates the race at maximum speed and exports an `.npz` file, while `viz_replay.py` renders the race with interactive controls.

1. Record an episode

```
# Solo run (default random driver, 2 laps)
docker compose run --rm recorder

# Custom run (name, cars, laps)
docker compose run --rm recorder python sim_recorder.py --steps 1000 --num-cars 2 --out
/app/my_race.npz

# Multi-car recording (up to 4 cars)
docker compose run --rm recorder python sim_recorder.py --steps 1000 --num-cars 4

# Multi-car on a specific map
docker compose run --rm recorder python sim_recorder.py --steps 2000 --num-cars 4 --laps 2 --map
Spa --out /app/spa_grand_crash.npz

# With your trained submission
docker compose run --rm recorder python sim_recorder.py --steps 1000 --submission submission/
```

2. Replay recorded episodes (requires display)

```
# Interactive replay of the latest episode (requires X11)
xhost +local:docker
docker compose run --rm -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix replay

# Replay a specific episode with camera following car 0
docker compose run --rm -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix replay python
viz_replay.py --npz /app/my_race.npz --follow
```

Replay controls

Key / Action	Description
Space / Play / Pause	Play / pause in real time
← / →	Step backward / forward 1 frame
Shift+← / Shift+→	Step backward / forward 10 frames
Slider	Jump to any step
Mouse Wheel	Zoom in / out centered on mouse cursor

{EPITECH}

Key / Action	Description
Click + Drag	Pan across the circuit
Double-Click / `R`	Reset camera and zoom

Export MP4 video Record the full replay as an MP4 video (headless, no display needed):

```
# Export the latest episode as MP4
docker compose run --rm replay python viz_replay.py --record /app/video.mp4 --no-telemetry

# Export with a specific episode
docker compose run --rm replay python viz_replay.py --npz /app/my_race.npz --record
/app/video.mp4 --no-telemetry

# Or locally (headless, no display)
python3 resources/4students/simulation/viz_replay.py --npz
resources/4students/simulation/recordings/episode_0001.npz --record output.mp4 --no-telemetry
```

The video plays at simulation speed (20fps = `sim_dt`).



🏋️ Training Your Agent

You create your own training pipeline (`train.py`, `env.py`, etc.).

The image ships what the simulation and the submission need, and nothing else: `numpy`, `numba`, `scipy`, `Pillow`, `pyyaml`, `imageio`, `matplotlib`, `tensorboard`, `pytest`, plus the pinned ONNX toolchain `torch` / `onnx` / `onnxscript` / `onnxruntime`.

Your training stack is yours to install — `gymnasium` for your `env.py`, and whatever framework you picked on top. Add it to the image (or to your own layer on top of it):

```
pip install "gymnasium>=1.0,<2.0" stable-baselines3
```

> Do not change the pinned `torch` / `onnx` / `onnxruntime` versions: they are what makes your exported model loadable by the tournament runtime.

`torch` is installed as a **CPU-only** build: the simulation needs no GPU, and the CUDA runtime would add ~2.7 GB to the image for nothing. If you want to train on a GPU, that is an opt-in on your side — reinstall `torch` from the CUDA index and expose your device to the container. A CPU-trained submission is perfectly acceptable.

```
# Run your training script
docker compose run --rm sim python train.py

# Launch TensorBoard in background (open http://localhost:6006)
docker compose up -d tensorboard
```

Submission structure

Local testing, a single agent To run `demo.py` or `sim_recorder.py` against your own agent, a flat `submission/` folder is enough: the loader treats it as a single team and replicates it across every car in self-play.

```
submission/
├── model.onnx
└── agent.py
```

Project delivery The delivery format is a different one: one subfolder per driver, plus a `champion` folder from Part 3 onwards. It is described in the project subject.



`agent.py` must declare an `Agent` class exposing `predict(obs: dict, info: dict) -> tuple[float, float]` returning `(target_speed, steering)`. (See the project subject for the full specification and the ONNX export constraints).

Vérifier une submission

`verify_submission.py` valide le format et le contenu d'une submission avant le tournoi. Deux modes :

Mode single agent (par défaut) : vérifie les imports, l'opset ONNX ≤ 17, le contrat `predict()`, le chargement via `agent_loader`. Les fichiers `agent.py`, `model.onnx` et si présent le `model.onnx.data` doivent être dans votre dossier `submission/`.

```
docker compose run --rm sim python verify_submission.py --submission submission/
```

Mode `--grandprix` : vérifie la structure de livraison (dossiers `driver1/`, `driver2/`, etc., chacun avec `agent.py` + `model.onnx`, noms uniques). Les fichiers de chaque voiture doivent être dans des sous-dossiers différents dans `submission/`.

```
docker compose run --rm sim python verify_submission.py --submission submission/ --grandprix
```


{EPITECH}

🗺️ Switching Circuits / Maps

26 circuits are available under `maps/<Name>/` : Austin, BrandsHatch, Budapest, Catalunya, Hockenheim, IMS, Melbourne, MexicoCity, Montreal, Monza, MoscowRaceway, Nuerburgring, Oschersleben, Sakhir, SaoPaulo, Sepang, Shanghai, Silverstone, Sochi, Spa, Spielberg, YasMarina, Zandvoort, example.

```
from env_simulation import set_map, get_available_maps, reset, get_obs, apply_action,
simulation_step, get_step_info, close

# List all available tracks
print(get_available_maps())

# Switch active track
set_map("Spa")

# Run simulation on the selected map
reset(num_cars=1)
for _ in range(500):
    obs = get_obs(0)
    apply_action(0, 3.0, 0.0)
    simulation_step()
    info = get_step_info()
    if info["lap_complete"][0]:
        break
close()
```

📋 Arguments Reference

`sim_recorder.py`

Argument	Default	Description
`--steps N`	`2000`	Maximum decision steps
`--laps N`	`2`	Target completed laps per car
`--num-cars N`	`1`	Number of cars on track (1–4)
`--map NAME`	`example`	Map/track name (Austin, BrandsHatch, Budapest, Catalunya, Hockenheim, IMS, Melbourne, MexicoCity, Montreal, Monza, MoscowRaceway, Nuerburgring, Oschersleben, Sakhir, SaoPaulo, Sepang, Shanghai, Silverstone, Sochi, Spa, Spielberg, YasMarina, Zandvoort)
`--submission PATH`	`""`	Path to a submission directory or parent folder of team subdirectories
`--model PATH`	`""`	Direct path to an ONNX model file for car 0
`--controller {auto,submission,random,onnx,pure_pursuit}`	`auto`	Policy controller type
`--out PATH`	auto	Output `.npz` file path (auto: `recordings/episode_XXXX.npz`)

`viz_replay.py`

Argument	Default	Description
`--npz PATH`	`""`	Path to `.npz` recording file
`--follow`	`false`	Follow car 0 with camera
`--record PATH`	`""`	Export replay as MP4 video (headless)
`--telemetry`	`true`	Show telemetry overlay (default: on)
`--no-telemetry`	`false`	Hide telemetry overlay for cleaner video
`--no-display`	`false`	Headless mode (saves snapshot PNG)




EPITECH